

Multiplayer Session-based Game Hosting on AWS

Architecture Diagrams Game Tech

[Home \(multiplayer-session-based-game-hosting-on-aws.html\)](#)

Multiplayer Session-based Game Hosting on AWS

[PDF \(/pdfs/architecture-diagrams/latest/multiplayer-session-based-game-hosting-on-aws/multiplayer-session-based-game-hosting-on-aws.pdf#multiplayer-session-based-game-hosting-on-aws\)](#)

[RSS \(multiplayer-session-based-game-hosting-on-aws.rss\)](#)

Publication date: **September 1, 2022**
([Diagram history \(#diagram-history\)](#))

This architecture enables you to use Amazon GameLift multi-Region fleets and a serverless backend solution to host a session-based multiplayer game.

On this page

[Multiplayer Session-based Game Hosting on AWS Diagram \(#diagram1\)](#)

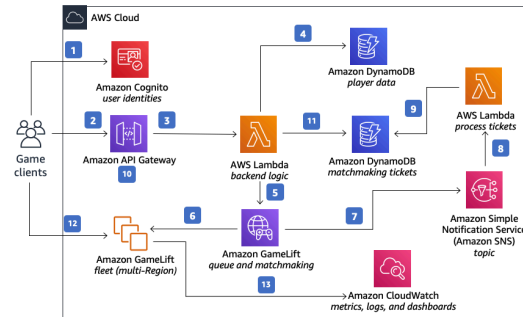
[Download editable diagram \(#download-editable-diagram\)](#)

[Create a free AWS account \(#create-a-free-aws-account\)](#)

[Further reading \(#further-reading\)](#)

[Diagram history \(#diagram-history\)](#)

Multiplayer Session-based Game Hosting on AWS Diagram



1. The game client requests an **Amazon Cognito** identity and temporary AWS credentials.
2. The client signs a matchmaking request to **API Gateway** with the temporary credentials. The request includes client latency information to supported AWS Regions.
3. **API Gateway** calls an **AWS Lambda** function with player identity information.
4. The **Lambda** function gets the player skill level from a **DynamoDB** table.
5. The **Lambda** function requests matchmaking from **Amazon**

GameLift FlexMatch with player skill and latency data.

6. **Amazon GameLift FlexMatch** creates a match with multiple players, and an **Amazon GameLift** queue allocates a session in an **Amazon GameLift** fleet location based on the latency data.
7. **Amazon GameLift FlexMatch** publishes an event to **Amazon SNS** on matchmaking success.
8. **Amazon SNS** triggers a subscribed **Lambda** function for ticket processing.
9. The **Lambda** function stores the ticket result in a **DynamoDB** table.
10. The game client polls for matchmaking success on a defined interval from **API Gateway**.
11. The **Lambda** function checks matchmaking information from the **DynamoDB** table and informs the client of a successful match by returning server IP, port, and player session ID.
12. The client connects directly to the server and sends the player session ID. The **Amazon GameLift Server SDK** is used to validate the player session.
13. Game servers send logs and metrics to **Amazon CloudWatch** with the **CloudWatch** agent.

Download editable diagram

To customize this reference architecture diagram based on your business needs, [download the ZIP file \(samples/multiplayer-session-based-game-hosting-on-aws.zip\)](#) which contains an editable PowerPoint.

Create a free AWS account

Sign up now



(<https://portal.aws.amazon.com/gp/aws/developer/registration/index.html>)

Sign up for an AWS account. New accounts include 12 months of [AWS Free Tier](#) (<https://aws.amazon.com/free/>) access, including the use of Amazon EC2, Amazon S3, and Amazon DynamoDB.

Further reading

For additional information, refer to

- [AWS Architecture Icons](#) (<https://aws.amazon.com/architecture/icons>)

- [AWS Architecture Center](https://aws.amazon.com/architecture)
(<https://aws.amazon.com/architecture>)
- [AWS Well-Architected](https://aws.amazon.com/architecture/well-architected)
(<https://aws.amazon.com/architecture/well-architected>)

Diagram history

To be notified about updates to this reference architecture diagram, subscribe to the RSS feed.

Change	Description	Date
Initial publication	Reference architecture diagram first published.	September 1, 2021

Note

To subscribe to RSS updates, you must have an RSS plugin enabled for the browser you are using.